



in28minutes

Getting Started with Gradle Build Tool



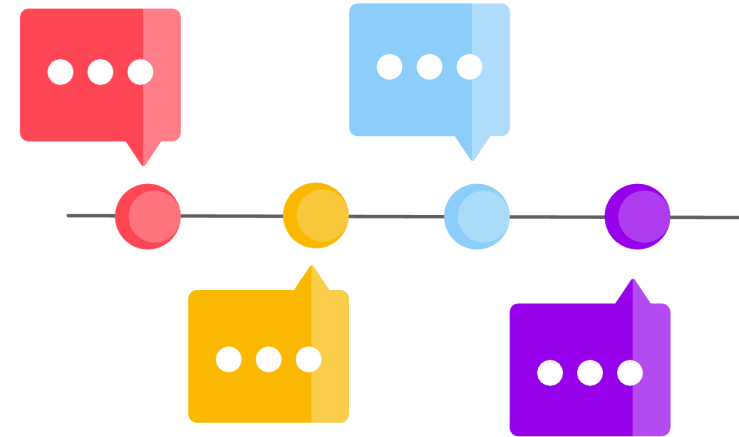
The Context: Evolution of Build Tools

Ant: Procedural (Do A, then B). XML based. Very flexible but verbose. No dependency management.

Maven: Declarative (I want X). XML based. Standardized. "Opinionated".

The Problem: What if you need the *Structure* of Maven but the *Flexibility* of Ant?

Enter Gradle: The best of both worlds.





Enter Gradle: Performance & Flexibility

DSL (Domain Specific Language):

- Based on **Groovy** or **Kotlin**
- **Code**, not just configuration

High Performance:

- **Incremental Builds:** Only rebuilds what changed
- **Build Cache:** Reuses outputs from previous builds

Same project layout: As Maven

IDE support: Still evolving





Gradle Core Concept 1: build.gradle and settings.gradle

Clean & Concise: Compare this to `pom.xml`. No angle brackets!

Readable: Reads almost like English sentences.

Configuration:

- `group`: The organization
- `settings.gradle` -> `rootProject.name = 'learn-gradle'`: The project name
- `version`: The specific release
- `repositories`: Maven Central Repo
- `dependencies`: Your dependencies

```
group = 'com.in28minutes'
version = '0.0.1-SNAPSHOT'

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-
starter-webmvc'
    testImplementation 'org.springframework.boot:spring-boot-
starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```



Gradle Core Concept 2: Plugins

Extensibility: Gradle core is small.
Everything else is a **Plugin**.

- **Java Plugin:** Adds tasks - `compileJava`, `test`, `jar`.
 - **Default Layout:** Defines default layout
- **Spring Boot Plugin:** Adds `bootRun`, `bootJar`, `bootBuildImage` and imports `spring-boot-dependencies`
- **Dependency Management Plugin:** Maven-like dependency management
 - `group:'org.springframework', name:'spring-core', version:'10.0.3'` OR
 - Shortcut: `org.springframework:spring-core:10.0.3` OR `org.springframework:spring-core(dependency management enabled by spring-boot-dependencies)`

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version 'x.y.z'  
    id 'io.spring.dependency-management'  
    version 'a.b.c'  
}
```



Gradle Core Concept 3: Tasks

Everything is a Task:

- `compileJava` is a task
- `compileTestJava` is a task
- `test` is a task
- `build` is a task

Task Graph: Gradle calculates the most efficient order to run tasks

Custom Tasks: Define custom tasks

```
tasks.named('test') {  
    useJUnitPlatform()  
}  
  
task hello {  
    doLast {  
        println 'Hello, in28minutes!'  
    }  
}
```



Maven vs Gradle: The Showdown

Feature	Maven	Gradle
Configuration	XML (Declarative)	Groovy/Kotlin (Programmable)
Flexibility	Low (Strict Standards)	High (Do anything)
Performance	Good	Excellent (Incremental, Cache)
Learning Curve	Easy (Standard)	Steeper (Need to learn DSL)
Use Case	Standard Java Projects	Android, Complex Builds, Monorepos



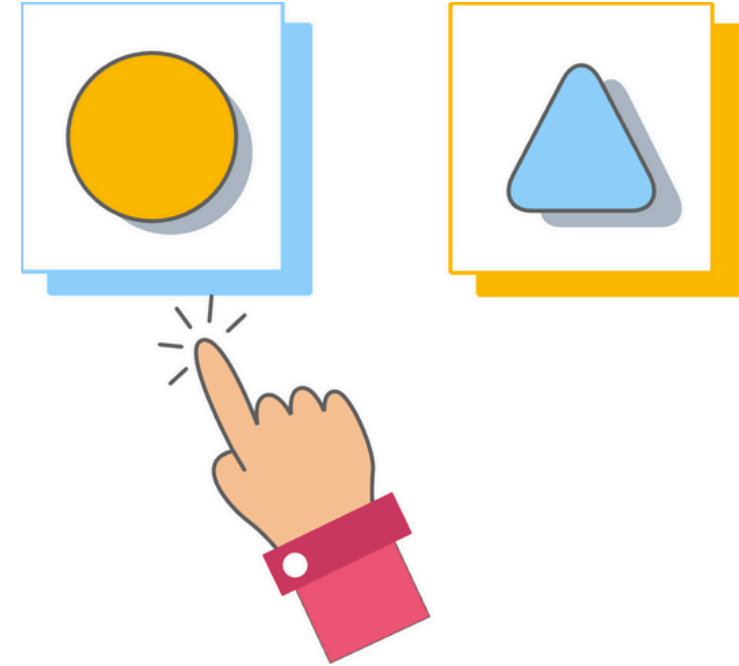
Maven vs Gradle: Which one to use

Let's start: With a few popular examples

- **Spring Framework** - Using Gradle since 2012 (v3.2.0)
- **Spring Boot** - Using Gradle since 2020 (v2.3.0)
- **Spring Cloud** - Continues to use Maven even today
 - Last update: Spring Cloud has no plans to switch
- **Top Maven Advantages:** Simple, Familiar, Restrictive
- **Top Gradle Advantages:** Fast builds, less verbose

What I Recommend:

- Choose whatever tool best meets your projects needs
- If your builds are taking really long, go with Gradle
- If your builds are simple, stick with Maven





Summary

Gradle brings **Flexibility** and **Performance**

- Uses **Groovy** or **Kotlin** DSL instead of XML
- **Incremental Builds** make it significantly faster than Maven for large projects

Tasks are the unit of work

Plugins provide the capabilities

Android builds use Gradle almost exclusively

